MISR UNIVERSITY
FOR SCIENCE & TECHNOLOGY
College of Information Technology

جامعة مصر
للعلوم والتكنولوجيا
كلية تكنولوجيا المعلومات

Course Name:

Image Processing

Course code:

CS.383

Project Title:

Image Processing Pipeline for Trash Detection and Scene Segmentation.

Submitted to:

T.A. Shahd Hatem.

Team members

Name	ID
Amal Alhaj	200039157
Salma Etman	200037699
Tasneem Khatery	200053109

Abstract:

This project focuses on building an image processing pipeline for trash detection and automated scene cleanup using classical computer vision techniques. The system works on a real dataset Kaggle of waste images and applies a series of operations including preprocessing, filtering, edge detection, and segmentation to analyze and enhance image content. The goal is to prepare images for further automated cleanup tasks by identifying and separating relevant objects from the background without using deep learning models.

Problem definition:

Urban environments often suffer from visual pollution caused by scattered trash and unwanted objects in public scenes. Manual cleanup and annotation of such images is time-consuming and inefficient. Additionally, many automated solutions rely on deep learning models, which require heavy computation and training data.

Therefore, there is a need for a lightweight and efficient image processing approach that can:

- Collect and organize image data.
- Preprocess raw images for consistency.
- Enhance image quality through filtering.
- Detect edges and object boundaries.
- Segment regions of interest from the background.

Problem solution:

The proposed system uses a classical computer vision pipeline implemented with OpenCV. The process includes:

1. **Image Dataset Collection:**
   A real dataset is loaded and organized for processing.
2. **Preprocessing:**
   Images are converted to grayscale and resized to ensure uniformity.
3. **Image Filtering:**
   Noise is reduced using Gaussian, Median, and Bilateral filters to improve image quality.

4. **Edge Detection:**
   Canny and Sobel operators are used to extract object boundaries and structural details.

5. **Segmentation:**
   Thresholding techniques (binary and adaptive) are applied to separate foreground objects from the background.

6. **Visualization:**
   Each stage is displayed to analyze how the image transforms through the pipeline.

This approach provides a simple, fast, and interpretable solution for scene analysis without relying on neural networks.

Code:

```
from google.colab import files

uploaded = files.upload()
```

```
!pip install opencv-python matplotlib pillow numpy
```

```
import os
import cv2
import glob
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

```
!apt-get install -y p7zip-full

!7z x /content/archive.zip -o/content/dataset
```

```
!ls /content/dataset
```

```
DATASET_PATH = "/content/dataset"

IMAGE_PATH = os.path.join(DATASET_PATH, "test/images")

images = glob.glob(os.path.join(IMAGE_PATH, "*.jpg"))

print("Total Images:", len(images))

for i, img in enumerate(images[:5]):
    print(f"[{i+1}] {img}")
```

```python
def load_image(image_path):

    img_bgr = cv2.imread(image_path)

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    return img_bgr, img_rgb
```

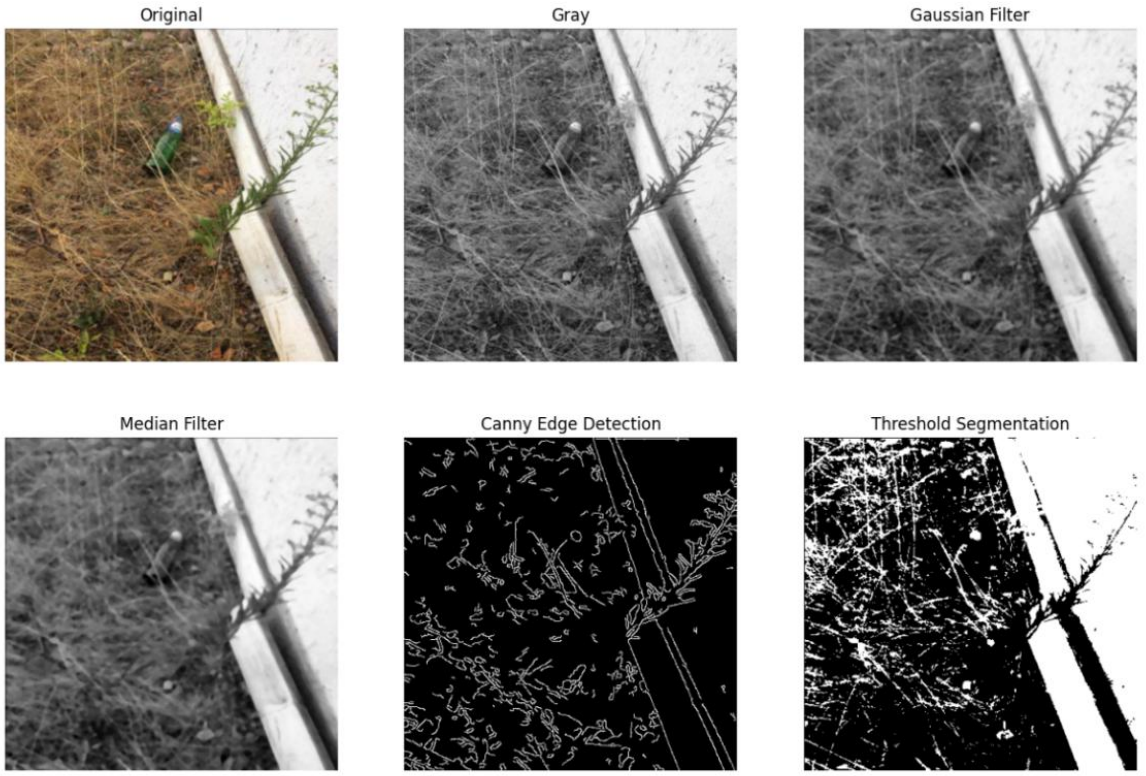
```python
def preprocess(img_bgr):

    # Convert to grayscale
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # Resize image
    resized = cv2.resize(gray, (640, 640))

    return gray, resized
```

```python
def filtering(gray):

    # Gaussian Filter
    gaussian = cv2.GaussianBlur(gray, (5, 5), 0)

    # Median Filter
    median = cv2.medianBlur(gray, 5)

    # Bilateral Filter
    bilateral = cv2.bilateralFilter(gray, 9, 75, 75)

    return gaussian, median, bilateral
```

```python
def edge_detection(filtered):

    # Canny Edge Detection
    canny = cv2.Canny(filtered, 100, 200)

    # Sobel Edge Detection
    sobelx = cv2.Sobel(filtered, cv2.CV_64F, 1, 0, ksize=5)

    sobely = cv2.Sobel(filtered, cv2.CV_64F, 0, 1, ksize=5)

    sobel = cv2.magnitude(sobelx, sobely)

    return canny, sobel
```

```python
def segmentation(gray):

    # Binary Threshold
    _, thresh = cv2.threshold(
        gray,
        127,
        255,
        cv2.THRESH_BINARY
    )

    # Adaptive Threshold
    adaptive = cv2.adaptiveThreshold(
        gray,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11,
        2
    )

    return thresh, adaptive
```

```python
def show_results(
    original,
    gray,
    gaussian,
    median,
    edges,
    thresh
):

    plt.figure(figsize=(15,10))

    # Original
    plt.subplot(2,3,1)
    plt.title("Original")
    plt.imshow(original)
    plt.axis("off")

    # Gray
    plt.subplot(2,3,2)
    plt.title("Gray")
    plt.imshow(gray, cmap='gray')
    plt.axis("off")

    # Gaussian
    plt.subplot(2,3,3)
    plt.title("Gaussian Filter")
    plt.imshow(gaussian, cmap='gray')
    plt.axis("off")

    # Median
    plt.subplot(2,3,4)
    plt.title("Median Filter")
    plt.imshow(median, cmap='gray')
    plt.axis("off")

    # Edges
    plt.subplot(2,3,5)
    plt.title("Canny Edge Detection")
    plt.imshow(edges, cmap='gray')
    plt.axis("off")

    # Segmentation
    plt.subplot(2,3,6)
    plt.title("Threshold Segmentation")
    plt.imshow(thresh, cmap='gray')
    plt.axis("off")

    plt.show()
```

```python
image_path = images[0]

# Load image
img_bgr, img_rgb = load_image(image_path)

# Preprocessing
gray, resized = preprocess(img_bgr)

# Filtering
gaussian, median, bilateral = filtering(gray)

# Edge Detection
edges, sobel = edge_detection(gaussian)

# Segmentation
thresh, adaptive = segmentation(gray)

# Show Results
show_results(
    img_rgb,
    gray,
    gaussian,
    median,
    edges,
    thresh
)
```

Output: